Document number	P3824R1
Date	2025-10-4
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Core Working Group Evolution Working Group SG23: Safety and Security

Static storage for braced initializers NBC examples

Table of contents

Changelog

R1

- made inline references more pronounced
- added `Wording` section

Abstract

Require `static storage duration` in `Static storage for braced initializers` [1] so that more memory unsafety will not be added to the `C++` language and thus ensure that this feature will be utilized instead of being bypassed for legacy code that does offer such assurances.

Motivation

The problem

Without `Static storage for braced initializers` [1:1] the programmer has to perform a work around as stated in `2.1. Workarounds` section. [2]

> *"This code creates a 2MB backing array on the stack frame of the function that initializes `v` :"*
>
> ```
> std::vector<char> v = {
>     #embed "2mb-image.png"
> };
> ```
>
> *"This code does not:"*
>
> ```
> static const char backing[] = {
>     #embed "2mb-image.png"
> };
> std::vector<char> v = std::vector<char>(backing, std::end(backing));
> ```
>
> *"So the latter is a workaround. But it shouldn't be necessary to work around this issue; we should fix it instead."*

Unfortuntely, with this feature, as currently worded, the programmer still has to do the work around.

`9.5.5 List-initialization [dcl.init.list] Paragraph 6` [3] makes no mention of `static storage duration` in the specification. It only has an example of what a compiler might do.

This is further confirmed in a footnote at the end of the standard.

> `C.1.4 Clause 9: declarations [diff.cpp23.dcl.dcl] Paragraph 2` [3:1]
>
> *"Permit the implementation to store backing arrays in static read-only memory."*

Permit is not a guarantee and consequently is less than the guarantee made in the workaround where the `static` keyword is used.

Without a requirement, this feature not only fails to provide the desired performance goals but also fails by increasing inconsistency, ambiguity and unsafety in the language. Consider the following example.

```
#include <initializer_list>

const char& first()
{
    std::initializer_list<char> il = { 'h', 'w' };
    return *il.begin();
}

const char& first(std::initializer_list<char> il)
{
    return *il.begin();
}

int main() {
    const char& rilm = first();
    const char& pilm = first({ 'h', 'w' });
    return 0;
}
```

This is actually two examples of potential dangling references, `rilm` and `pilm`, being created; both by return and by passthrough. If the compiler choose to make the lifetimes of the backing arrays of the initializer lists to have `static storage duration` than `rilm` and `pilm` are memory safe, race free and thread safe. If the compiler chooses `automatic storage duration` than `rilm` and `pilm` are dangling references.

If the programmer thinks that their code has `static storage duration` because it was based on the provided example in the standard and the compiler left it as `automatic storage duration` then the programmer likely did not make the necessary code changes to ensure it

doesn't dangle. Not being able to trust this feature will cause programmers to not trust their compilers and to program with the esoteric legacy workaround code, thus defeating the purpose of having this feature in the first place.

A programmer can't be expected to be responsible for dangling, if the programmer can't definitely reason about the lifetimes of objects. A code reviewer/code auditor can't be expected to find dangling, if the reviewer/auditor can't definitely reason about the lifetimes of objects. None of these three people should be forced to look at assembly, a different programming language, to know what the compiler decided to do in this instant and worse to repeat the process for each initializer list/braced initializer in a program.

Without definite verbiage in the standard, this is a non portable language feature between compilers. Worse yet, it may not be portable/stable within a single compiler as the behavior could vary with respect to different compiler flags or other internal logic.

Besides failing to require `static storage duration` the current verbiage fails to even mention the possibility of `static storage duration`.

> `9.5.5 List-initialization [dcl.init.list] Paragraph 6` [3:2]
>
> *"The backing array has the same lifetime as any other temporary object (6.8.7), except that initializing an initializer_list object from the array extends the lifetime of the array exactly like binding a reference to a temporary."*

Except that it is not exactly the same because when the lifetime of a temporary is extended it still has `automatic storage duration` changed from the statement to the block, like a declared variable.

C.1.4 Clause 9: declarations [diff.cpp23.dcl.dcl] Paragraph 2 [3:3]

```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

What is proposed is different because it is being extended by being turned into something that has `static storage direction`.

The solution

The lifetime of the backing array associated with initializer lists needs to definitely state `static storage duration` in a similar fashion as stated else where in the standard. Consider the very first occurence of `static storage duration` in the standard.

5.13.5 String literals [lex.string] Paragraph 9 [3:4]

"Evaluating a string-literal results in a string literal object with static storage duration (6.8.6)."

Should a initializer list of ints or even chars be specified radically nebulous from the clear straight forward verbiage of string literals. Fixing the initializer list verbiage with the string literal verbiage would make the two more consistent and in a reasonable fashion.

```
const char& first()
{
    const char* sl = "hw";
    return *sl;
}

const char& first(const char* sl)
{
    return *sl;
}

int main() {
    // no dangling
    const char& rslm = first();
    const char& pslm = first("hw");
    return 0;
}
```

```
#include <initializer_list>

const char& first()
{
    std::initializer_list<char> il = {
    return *il.begin();
}

const char& first(std::initializer_lis
{
    return *il.begin();
}

int main() {
    // no dangling
    const char& rilm = first();
    const char& pilm = first({ 'h', 'w
    return 0;
}
```

Even the example provided by the standard could benefit from some revisions.

`9.5.5 List-initialization [dcl.init.list] Paragraph 6` [3:5]

```
void f(std::initializer_list<double> il);
void g(float x) {
    f({1, x, 3});
}
void h() {
    f({1, 2, 3});
}
```

The current example could be expanded to better illustrate when this feature does kick in versus may kick in.

```
void f(std::initializer_list<double> il);
void g(float x) {
    f({1, x, 3});// automatic storage duration
}
void h() {
    f({1, 2, 3});// static storage duration
}
void i() {
    const float x = 2;
    // ...
    // ...
    // ...
    f({1, x, x, x, 3});// for now, may or may not be static storage duration
    // depending on how much local analysis a compiler is willing to do
    // hopefully mandated in future release, at least programmers
    // would still have the static storage duration guarantee with
    // f({1, 2, 2, 2, 3});
}
```

While it would be ideal if both function `h` and function `i` examples had `static storage duration`, since `f({1, x, x, x, 3});` requires analysis of additional lines of code it could make sense, on the short term, that it MAY have `static storage duration`. Hopefully, even this scenario would be strengthened in future `C++` releases since it is needed for the deduplication of constants in order to fulfill DRY, don't repeat yourself, and ODR, one definition rule. At least with requiring `static storage duration` on function `h`, their would be a fallback plan that would still allow the programmer to use this feature. Without even this minimal guarantee, programmers have to fallback on to esoteric workaround code.

Initializer lists are a pure reference type. Shouldn't other standard pure reference types also have similar functionality, such as `std::span<const T>` and `std::string_view`.

```
#include <span>

const char& first()
{
    //std::span<const char> il = {{ 'h', 'w' }};// pre C++26
    std::span<const char> il = { 'h', 'w' };// C++26
    return il.front();
}

const char& first(std::span<const char> il)
{
    return il.front();
}

int main() {
    const char& rilm = first();// may or may not dangle
    //const char& pilm = first({{ 'h', 'w' }});// pre C++26, dangle
    const char& pilm = first({ 'h', 'w' });// C++26, may or may not dangle
    return 0;
}



#include <initializer_list>
#include <string_view>

std::string_view to_string_view(std::initializer_list<char> il)
{
    return {il.begin(), il.size()};
}

const char& first()
{
    std::initializer_list<char> il = { 'h', 'w' };
    std::string_view sv = to_string_view(il);
    return sv.front();
}

const char& first(std::string_view il)
{
    return il.front();
}

int main() {
    const char& rilm = first();// may or may not dangle
    const char& pilm = first(to_string_view({ 'h', 'w' }));// may or may not dangl
    return 0;
}
```

Even these `std::span<const T>` and `std::string_view` examples would become safe if initializer lists were given a `static storage duration` guarantee.

The bonus problem

Another important issue that should be fixed is that the original proposal was for *"Static storage for **braced initializers**"* yet the current verbiage is only for initializer lists. If the currently worded functionality is reasonable even for an initializer list of size one, why not for single instance objects that are braced initialized entirely of constant literals.

`std::initializer_list<int> x1 = { 1 };`	`const int& x1 = {1};`
	`const int& x2{1};`
	`const int& x3 = 1;`

Might this be viewed as an inconsistency and even worse, a missed opportunity to fix a swath of dangling references by lifetime extending objects to have `static storage duration` .

`const int& first()`
`{`
`  const int& constant = 1;`
`  return cl;`
`  // safe if static`
`}`
	`const int& first()`
	`{`
	`  const int constant = 1;`
	`  return cl;`
	`}`
		`const int& firs`
		`{`
		`  return 1;`
		`  // safe if st`
		`}`

Example #2 Comments gcc, clang warning: returning reference to local temporary object [-Wreturn-stack-address] if not const error: non-const lvalue reference to type 'int' cannot bind to a temporary of type 'int' arguable should be error regardless of `const` `C++23` P2266R3 `Simpler implicit move` returning reference to xvalue is ill formed		

NOTE: The first and the third would be logically safe had the object had `static storage duration` . The second would be caught as an error if compilers such as clang and gcc would correctly implement/deploy `Simpler implicit move` [4]. If the bonus solution is accepted,

than hopefully future standards could require some local analysis so that the second could be made as safe as the first and third.

```
const int& first(const int& passthrough)
{
    return passthrough;
}

int main() {
    const int& x1 = first();// safe if static
    const int& x2 = first(1);// safe if static
    return 0;
}
```

Wording

Bonus wording

`6.8.7 Temporary objects [class.temporary] Paragraph 6 (6.8)` [3:6]

...

6 The third context is when a ~~reference~~pure alias type binds to a temporary object.[25]

Pure alias types are types that consist of only references, pointers and no value members other than the size of referenced memory. The set of pure alias types includes known pure alias types such as reference, pointer, `initializer_list`, `reference_wrapper`, `basic_string_view`, `span`, `mdspan`, `function_ref`, `optional<&>`, most if not all iterators, most if not all range views and aggregates of only pure alias types.

A type `is definitely NOT lifetime extendable` when extending the lifetime of said object would change the semantic meaning of a program. Known types that would fall into this category are locks, lock guards, files, sockets, transactions; pretty much any non memory only OS resource.

A type `is lifetime extendable` when it is `is_trivially_destructible` or the destructor just frees memory by calling `delete` or `free`. The type must not be or composed of types that are definitely NOT lifetime extendable. Known types that fall into this category are those

types that are `is_trivially_destructible`, also `Container` and `AssociativeContainer` when their contents are `lifetime extendable` and the collection itself doesn't have a functioning lock as that would make it definitely NOT lifetime extendable.

When the `pure alias type` is `const`, the type of the temporary `is lifetime extendable`, the temporary is being constructed via a `constexpr` or `consteval` constructor and all of the arguments to that constructor are entirely composed of `const` literals, then the lifetime of the temporary is static storage duration otherwise when a reference binds to a temporary object, t~~T~~he temporary object to which the reference is bound or the temporary object that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference if the glvalue to which the reference is bound was obtained through one of the following:

...

...

...

Change example #3

[Example 3 :

```
const int& x = (const int&)1; // temporary for value 1 has same lifetime as x
```

to

```
// temporary for value 1 is no longer temporary
// instead has lifetime of static storage duration
const int& x1 = (const int&)1;
const int& x2 = 1; // or simply
```

...

Initializer list only wording

`9.5.5 List-initialization [dcl.init.list] Paragraph 6` [3:7]

~~*"The backing array has the same lifetime*~~The backing array of an initializer_list has a lifetime of static storage duration unless one member of the array is initialized with a non `const` variable in which case the lifetime is the same *as any other temporary object (6.8.7), except that initializing an initializer_list object from the array extends the lifetime of the array exactly like binding a reference to a temporary."*

...

Impact on the standard

The requested wording changes has the same impacts as stated and accepted in the original feature. **C.1.4 Clause 9: declarations [diff.cpp23.dcl.dcl] Paragraph 2** [3:8]

The language impact of the original feature would mirror the impact of requiring the object created in the expression `const T& = T;` to have static storage duration. Namely, *"Valid C++ 2023 code that relies on the result of pointer comparison between ~~backing arrays~~objects may change behavior."* **C.1.4 Clause 9: declarations [diff.cpp23.dcl.dcl] Paragraph 2** [3:9]

The proposed changes are relative to the current working draft `N5014` [3:10].

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r3.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r3.html) ↩ ↩
2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r3.html#workarounds (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r3.html#workarounds) ↩
3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf) ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩
4. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html) ↩